

usuario:

pass:

DOCUMENTO FUNCIONAL:**CONTEXTO:**

Somos una Estudio que realizamos trabajos de Ingeniería de Datos a diferentes empresas.

Uno de nuestros clientes la empresa "CarsArgentina S.A." (concesionaria de vehículos) nos solicita resolver un problema de su negocio.

PROBLEMA DE LA EMPRESA:

La Gerencia de la empresa necesita tener información de los últimos meses (marzo, abril y mayo), para tomar decisiones de futuras incorporaciones de nuevas unidades para stock.

REQUERIMIENTOS DE LA EMPRESA:

La información requerida es la siguientes:

(Requerimiento 1) - Saber con datos concretos, las ganancias por mes y también el promedio de cada mes.

Solución en: {Notebook, celda: 38 } GRAFICO DE BARRAS.

(Requerimiento 2) - Cantidad de marcas de vehículos vendidos en esos meses con sus respectivos porcentajes.

Solución en: {Notebook, celda: 42 } GRAFICO DE TORTA

(Requerimiento 3) - Tendencia de evolucion de las ganancias mes a mes.

Solución en: {Notebook, celda: 44 } GRAFICO DE LINEAS

(Requerimiento 4) - Graficos representativos en cada caso.

(Requerimiento 5) - Tabla de datos resumida (Gold) para posteriores análisis que surjan.

Solución en: {Notebook, celda: 26 }

SOLUCIÓN TÉCNICA:

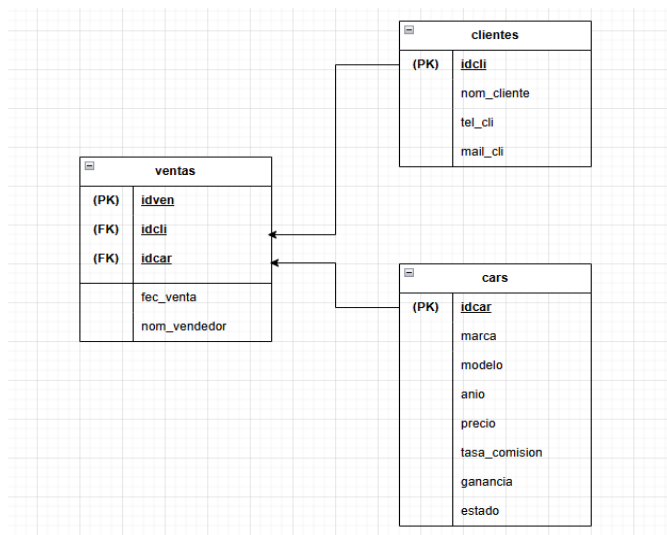
Vamos a utilizar Azure Data Factory como orquestador y Databricks como motor de transformación, respetando la arquitectura Medallion (Bronze, Silver, Gold).

>>> DATOS QUE VAMOS A USAR:

Archivos:

- ventas_data.csv (generada a partir de tablas maestras)
- ventas_data_suc.csv (generada a partir de tablas maestras)
- cliente.csv
- cars.csv
- ventas.csv

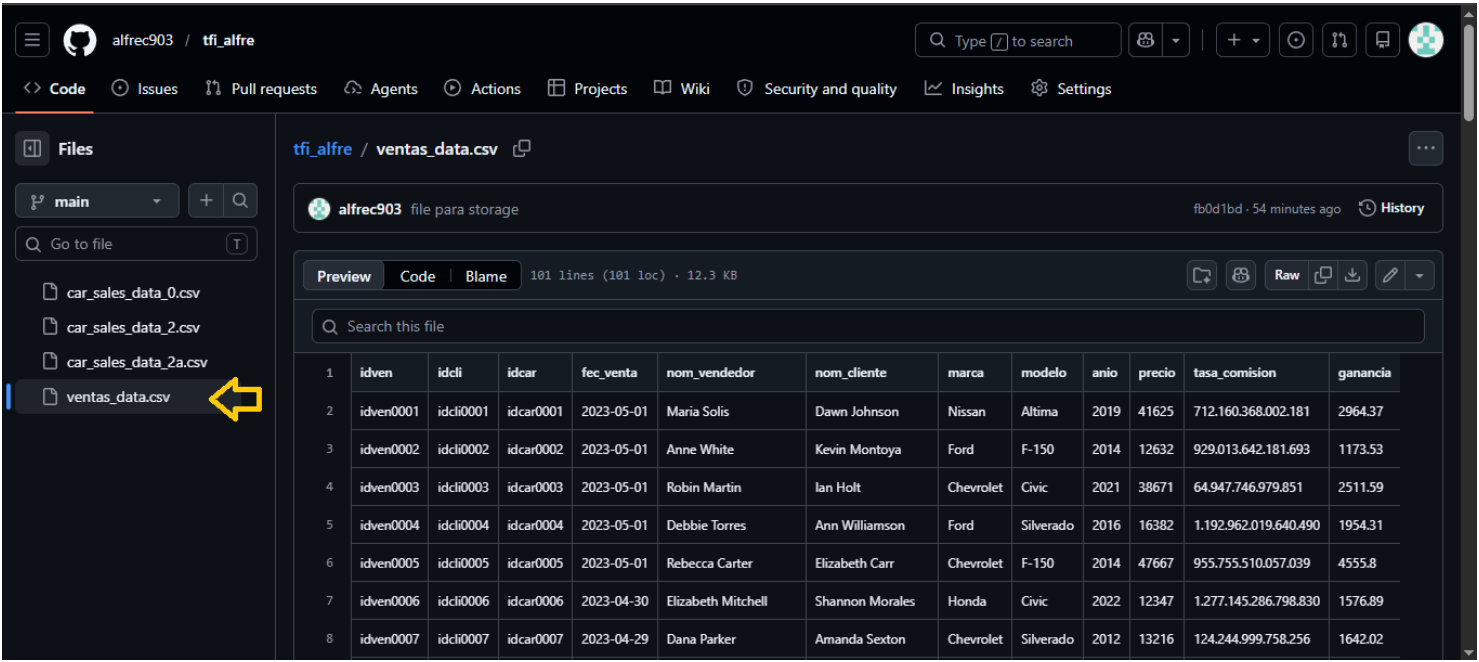
Img DER:



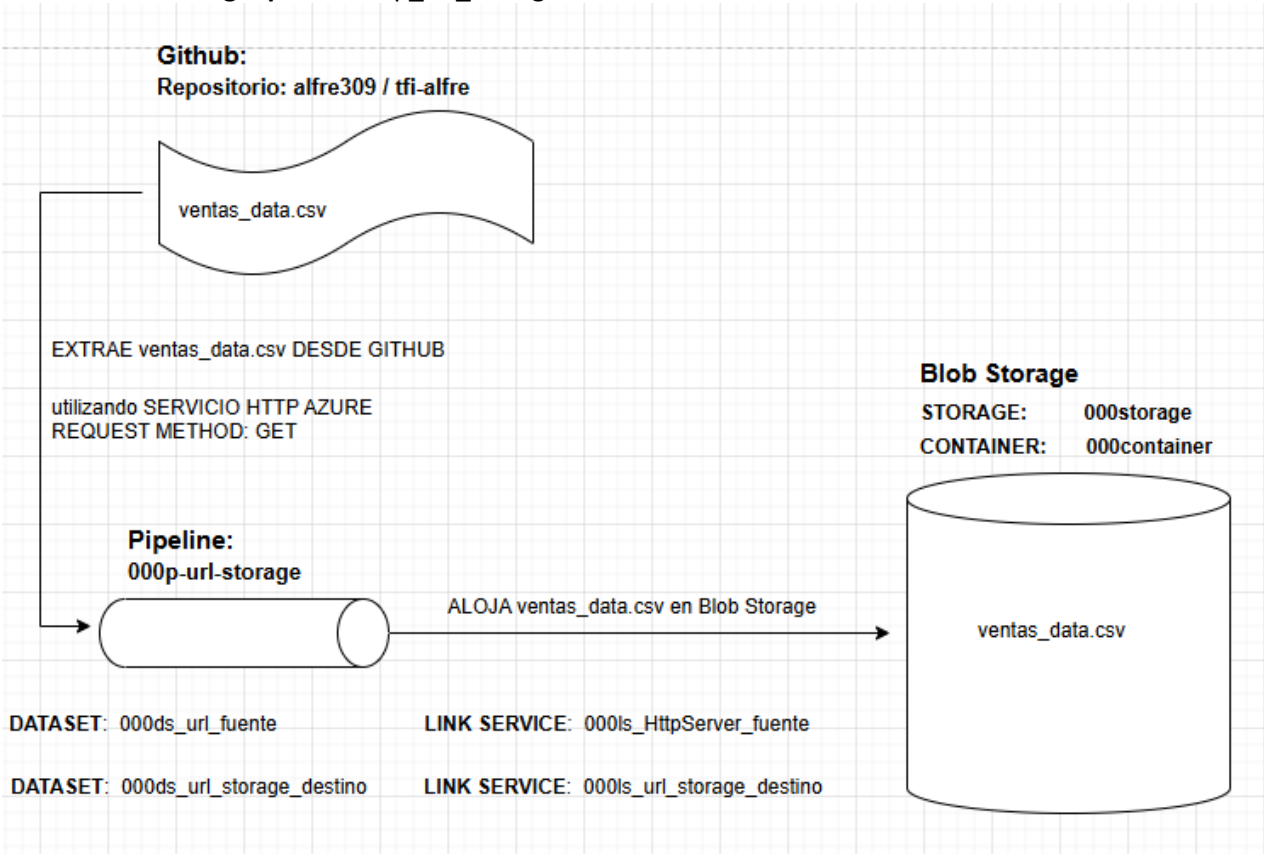
>>> **VAMOS A GENERAR DOS PIPELINE:**
PIPELINE 1)

000p_url_storage
Funcion: Va a copiar desde el repositorio Github/alfre903/tfi_alfre el archivo car_ventas_suc.csv y lo va almacenar en Blob Storage. Utilizando servicio http de azure.
Simulando Github ser una API de la sucursal de la empresa.
Imagenes representativa de dicha función:

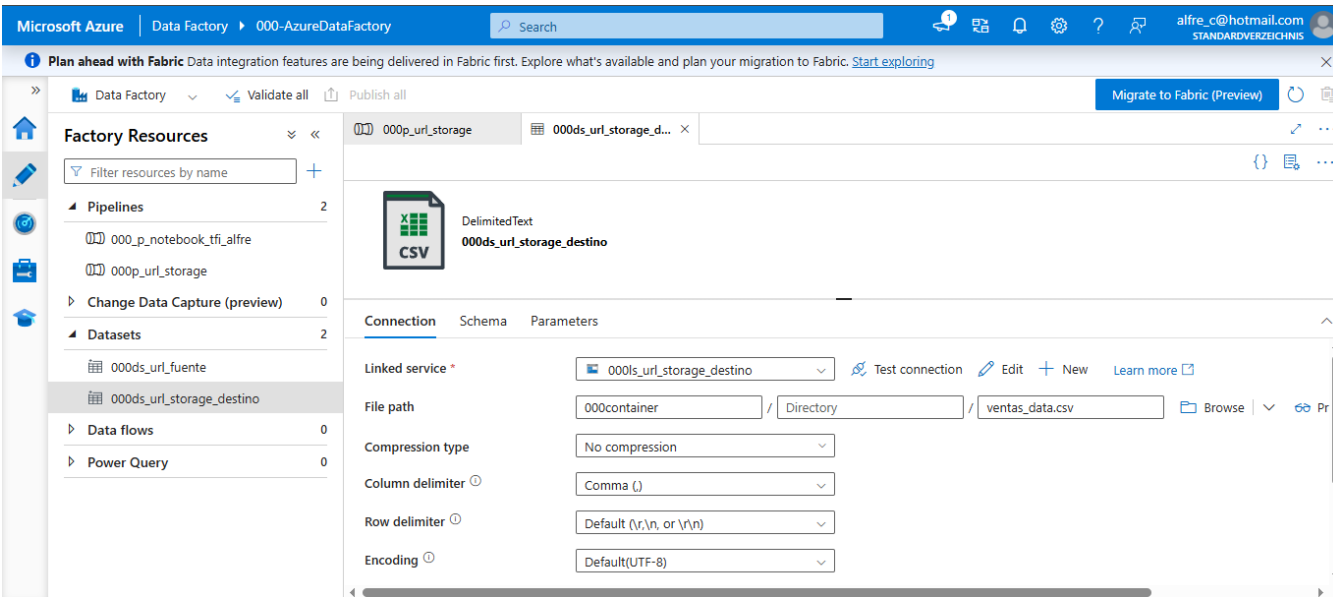
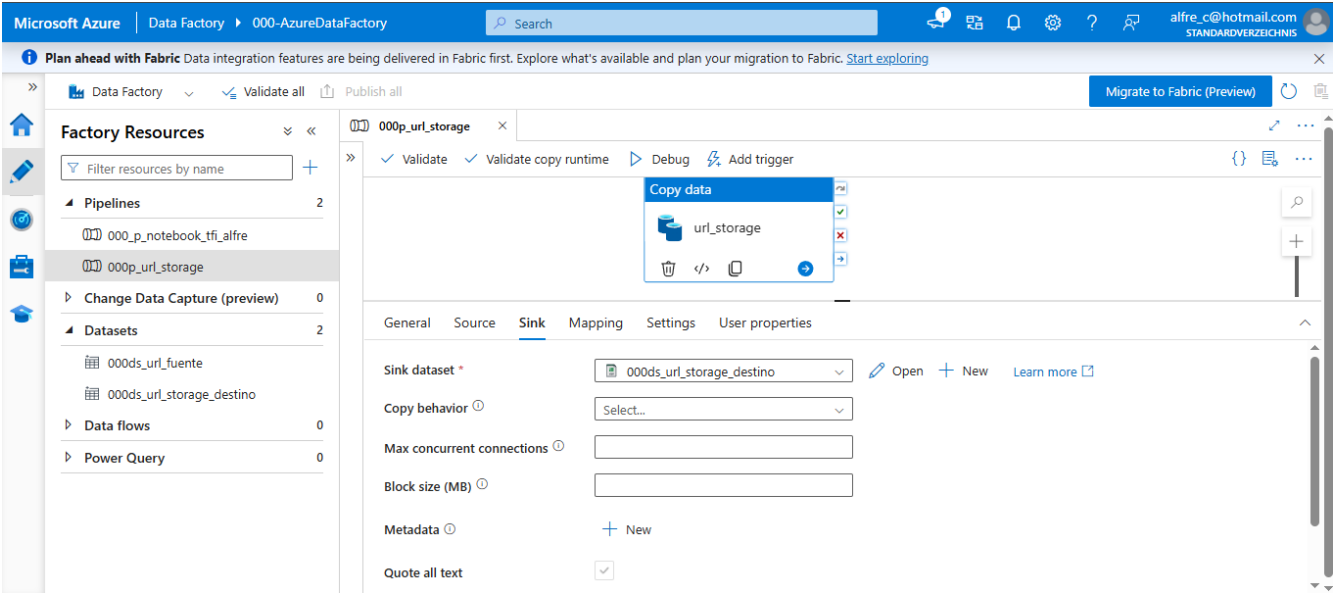
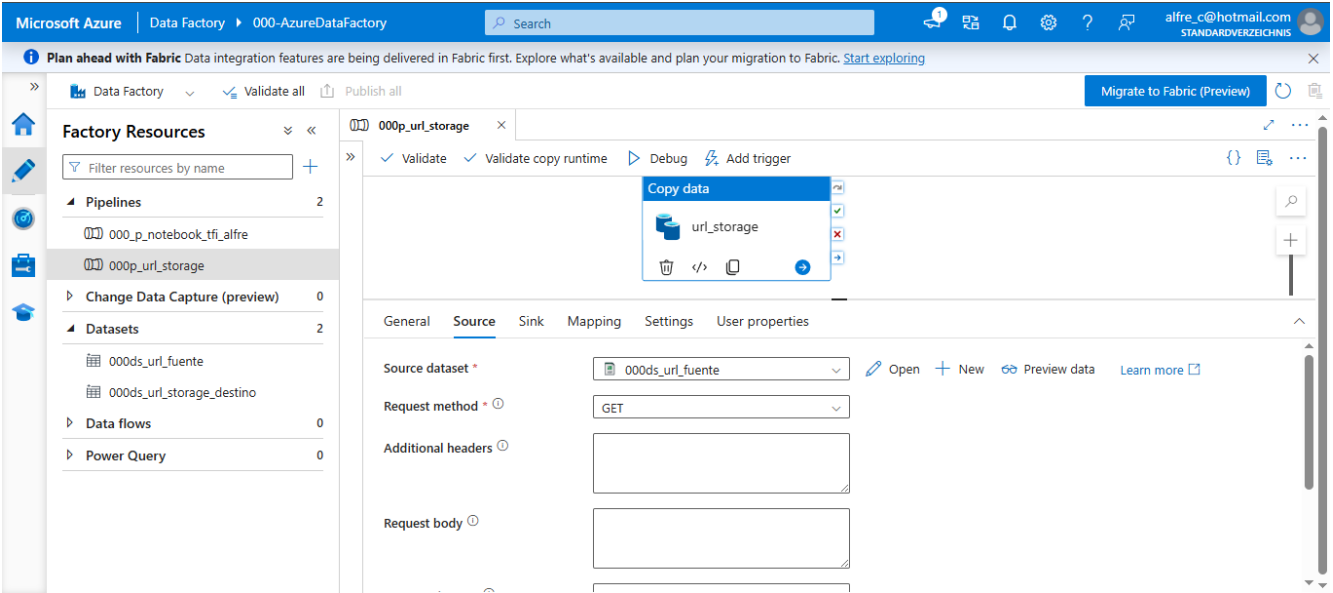
img Repositorio

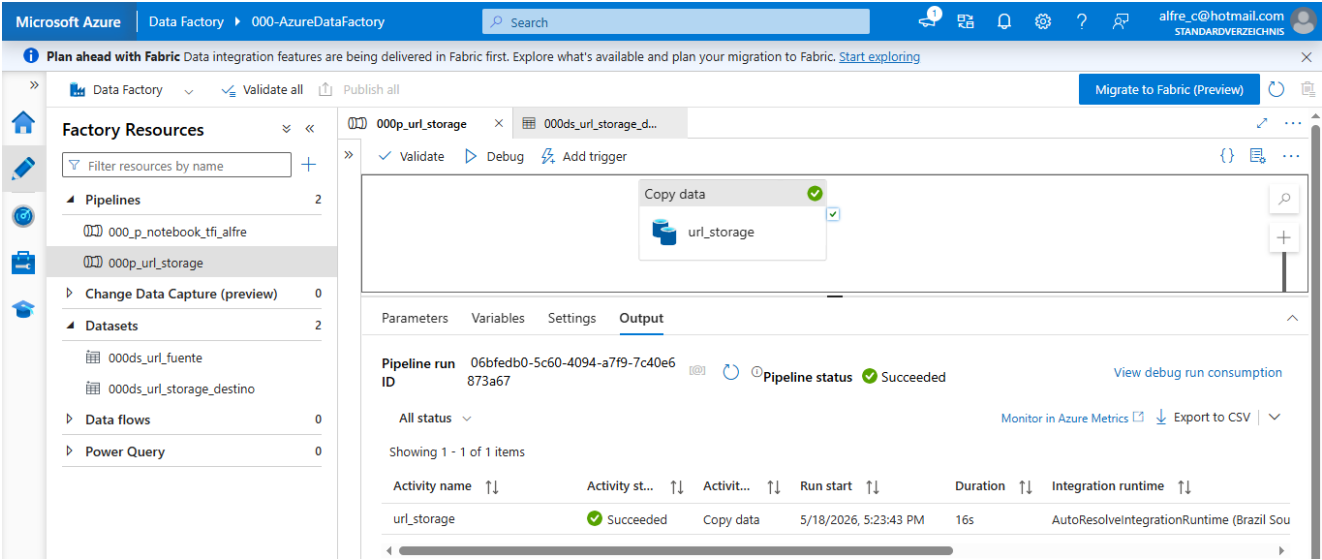


Img Pipeline 000p_url_storage



img capturas pipeline 000p_url_storage





Output de ejecución pipeline 000p_url_storage:

```
{
  "dataRead": 12599,
  "dataWritten": 12599,
  "filesRead": 1,
  "filesWritten": 1,
  "sourcePeakConnections": 1,
  "sinkPeakConnections": 1,
  "copyDuration": 11,
  "throughput": 6.299,
  "errors": [],
  "effectiveIntegrationRuntime": "AutoResolveIntegrationRuntime (Brazil South)",
  "usedDataIntegrationUnits": 4,
  "billingReference": {
    "activityType": "DataMovement",
    "billableDuration": [
      {
        "meterType": "AzureIR",
        "duration": 0.016666666666666666,
        "unit": "DIUHours"
      }
    ],
    "totalBillableDuration": [
      {
        "meterType": "AzureIR",
        "duration": 0.016666666666666666,
        "unit": "DIUHours"
      }
    ]
  },
  "usedParallelCopies": 1,
  "executionDetails": [
    {
      "source": {
        "type": "HttpServer"
      },
      "sink": {
        "type": "AzureBlobFS",
        "region": "Brazil South"
      },
      "status": "Succeeded",
      "start": "5/18/2026, 5:23:44 PM",
      "duration": 11,
      "usedDataIntegrationUnits": 4,
      "usedParallelCopies": 1,
      "profile": {
        "queue": {
          "status": "Completed",
          "duration": 7
        },
        "transfer": {
          "status": "Completed",
          "duration": 2,
          "details": {
            "listingSource": {
              "type": "HttpServer",
              "workingDuration": 0
            },
            "readingFromSource": {
              "type": "HttpServer",
              "workingDuration": 0
            },
            "writingToSink": {
              "type": "AzureBlobFS",
              "workingDuration": 0
            }
          }
        }
      },
      "detailedDurations": {
        "queuingDuration": 7,
        "transferDuration": 2
      }
    }
  ],
  "dataConsistencyVerification": {
    "VerificationResult": "Unsupported"
  },
  "durationInQueue": {
    "integrationRuntimeQueue": 0
  }
}
```

PIPELINE 2)

000_p_notebook_tfi_alfre

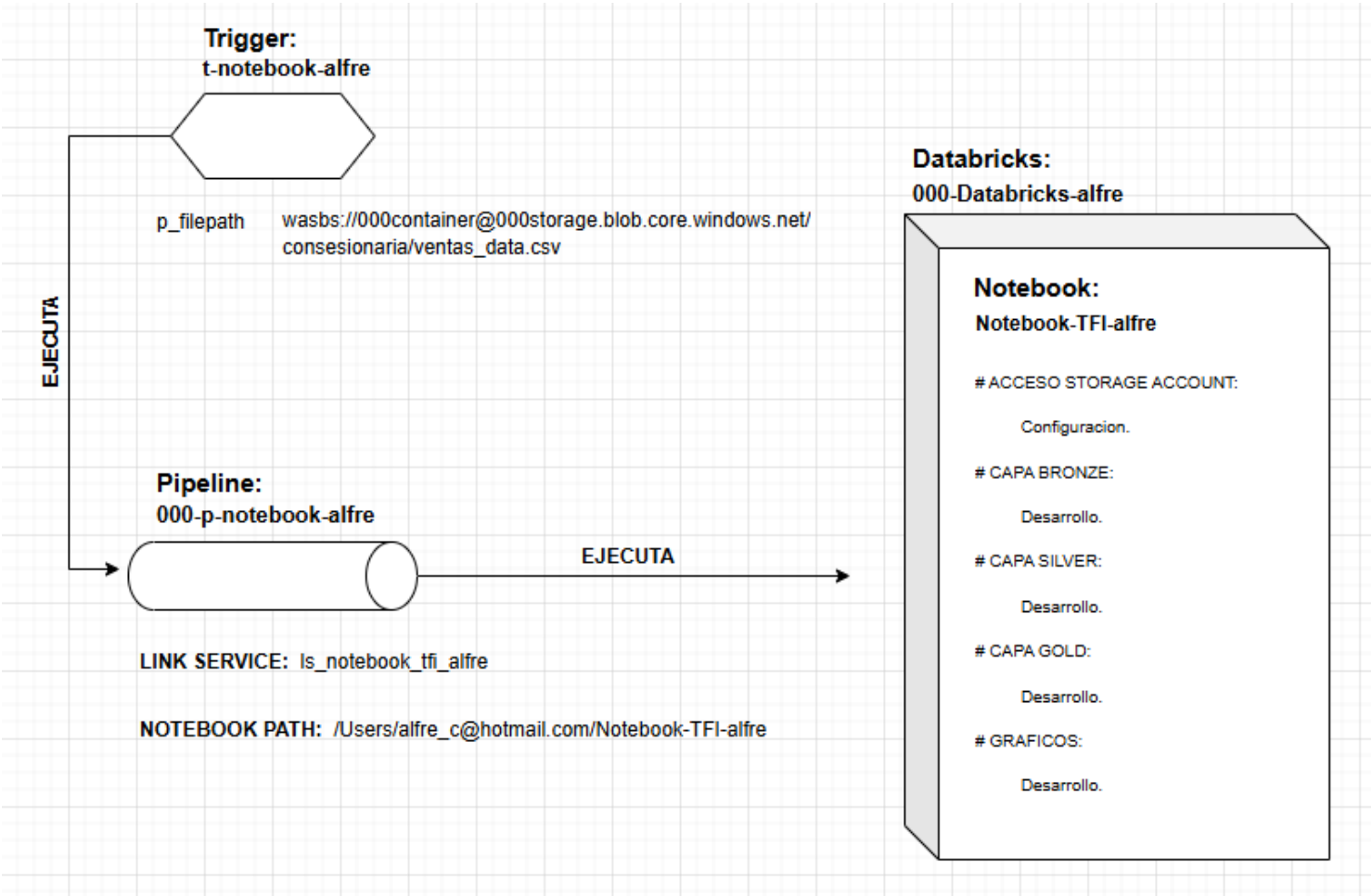
Funcion: Va a conectarse con Databricks que posee un Notebook (Notebook-TFI-alfre) para ejecutar los codigos que generan las Capas Bronze, Silver y Gold. Ademas de los Graficos.

Este pipeline tiene un Trigger (t-notebook-alfre) que conecta el notebook con los

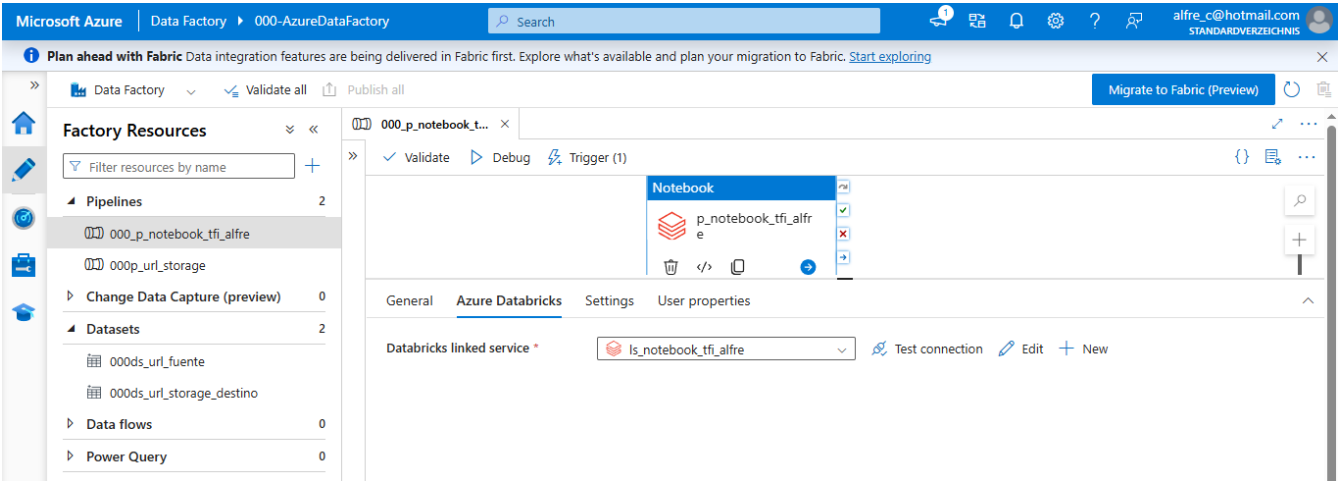
archivos de storage.

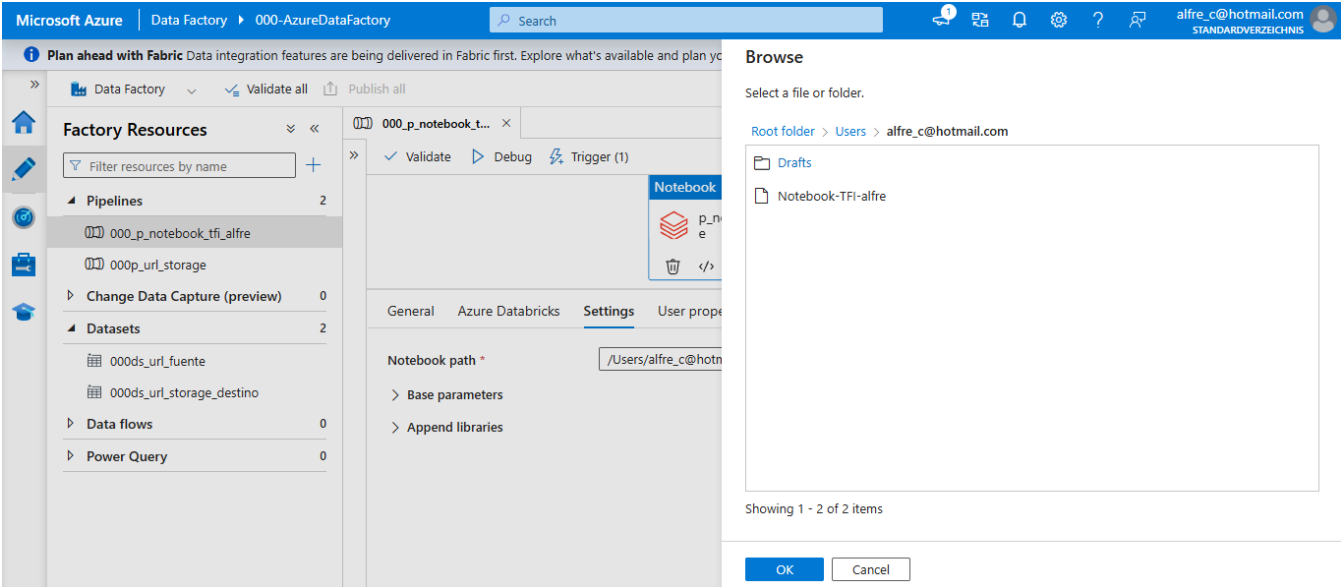
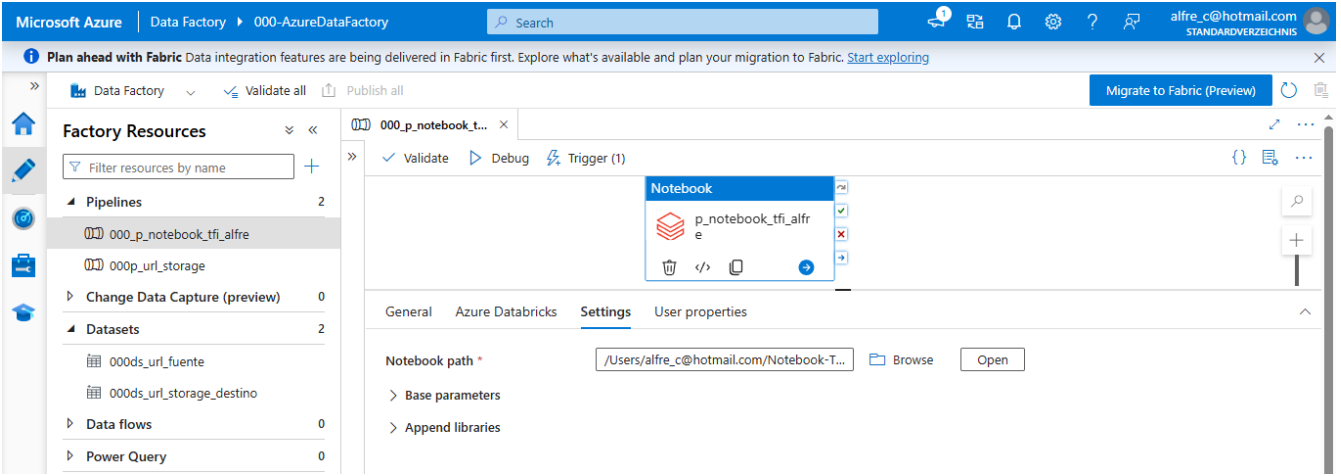
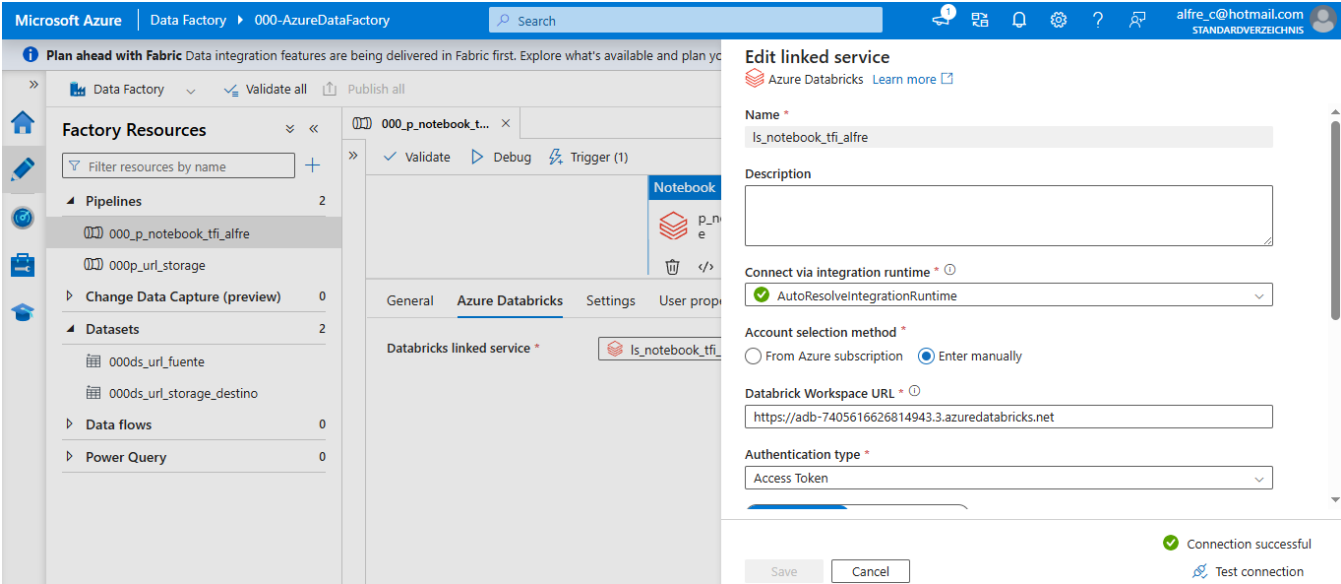
Imagen representativa de dicha función:

img pipeline 000_p_notebook_tfi_alfre



imgs capturas pipeline 000_p_notebook_tfi_alfre





Microsoft Azure | Data Factory | 000-AzureDataFactory

Plan ahead with Fabric Data integration features are being delivered in Fabric first. Explore what's available and plan your migration to Fabric. [Start exploring](#)

Factory Resources

- Pipelines: 2
 - 000_p_notebook_tfi_alfre
 - 000p_url_storage
- Change Data Capture (preview): 0
- Datasets: 2
 - 000ds_url_fuente
 - 000ds_url_storage_destino
- Data flows: 0
- Power Query: 0

Data preview

Make sure you have specific filters. Configuring filters that are too broad can match a large number of files created/deleted and may significantly impact your cost.

Event Trigger Filters

Container name: 000container
Starts with: conesionario/ventas_data
Ends with: .csv

1 blobs matched in "000container"

	Blob name
1	conesionario/ventas_data.csv

1 - 1 of 1 items

Continue Back Cancel

Microsoft Azure | Data Factory | 000-AzureDataFactory

Plan ahead with Fabric Data integration features are being delivered in Fabric first. Explore what's available and plan your migration to Fabric. [Start exploring](#)

Factory Resources

- Pipelines: 2
 - 000_p_notebook_tfi_alfre
 - 000p_url_storage
- Change Data Capture (preview): 0
- Datasets: 2
 - 000ds_url_fuente
 - 000ds_url_storage_destino
- Data flows: 0
- Power Query: 0

Output

Pipeline run ID: 38dd6775-6f47-4e38-9d71-b594e79ef701

Pipeline status: Succeeded

View debug run consumption

Monitor in Azure Metrics Export to CSV

Activity name	Activity status	Activity type	Run start	Duration	Integration runtime
p_notebook_tfi_alfre	Succeeded	Notebook	5/18/2026, 5:39:44 PM	7m 10s	AutoResolveIntegrationRuntime (Brazil South)

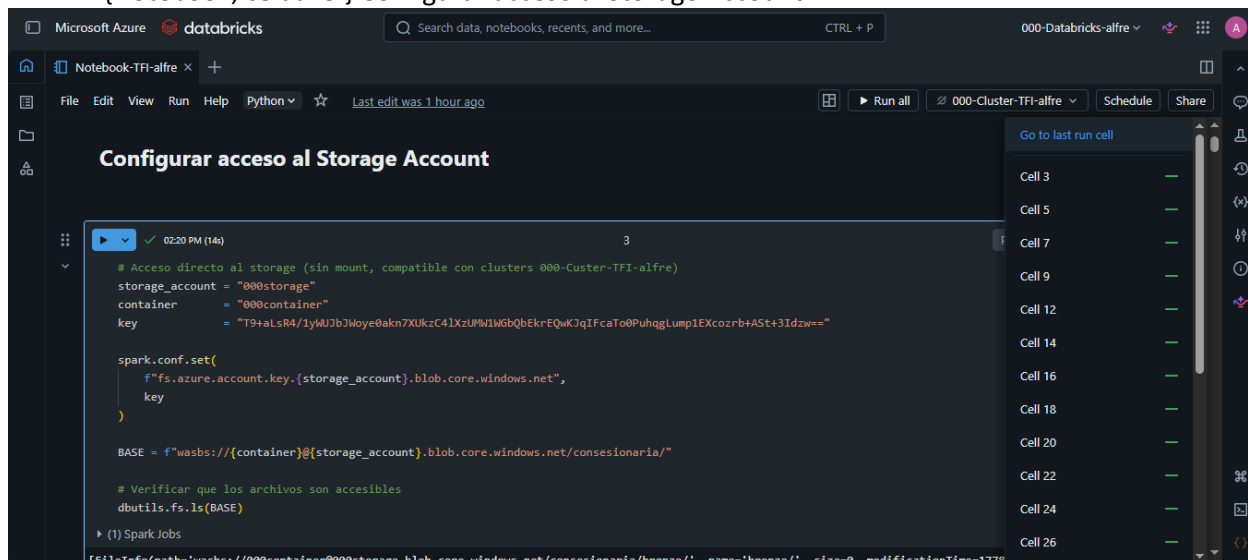
Output pipeline 000_p_notebook_tfi_alfre;

```
{
  "runPageUrl": "https://adb-7405616626814943.3.azuredatabricks.net/?o=7405616626814943#job/873851080809043/run/940203308360160",
  "effectiveIntegrationRuntime": "AutoResolveIntegrationRuntime (Brazil South)",
  "executionDuration": 424,
  "durationInQueue": {
    "integrationRuntimeQueue": 0
  },
  "billingReference": {
    "activityType": "ExternalActivity",
    "billableDuration": {
      "meterType": "AzureIR",
      "duration": 0.13333333333333333,
      "unit": "Hours"
    }
  }
}
```

***** Se genera un Databricks con un Notebook (Notebook-TFI-alfre):**

Funcion: Utilizando recursos de Azure como Cluster, lenguajes, frameworks y funciones, procedemos a codificar las siguientes bloques:

{Notebook, celda: 3 } Configurar acceso al Storage Account.



The screenshot shows the Databricks interface for a notebook named 'Notebook-TFI-alfre'. The code in cell 3 is as follows:

```
# Acceso directo al storage (sin mount, compatible con clusters 000-Cluster-TFI-alfre)
storage_account = "000storage"
container = "000container"
key = "T9+alsR4/1yMJb3Moye@akn7XUkzC41xzU%WlMGbQbEkrEQwKJqIFcaTo0PuhqgLump1EXcozrbtAST+3Idzw=="

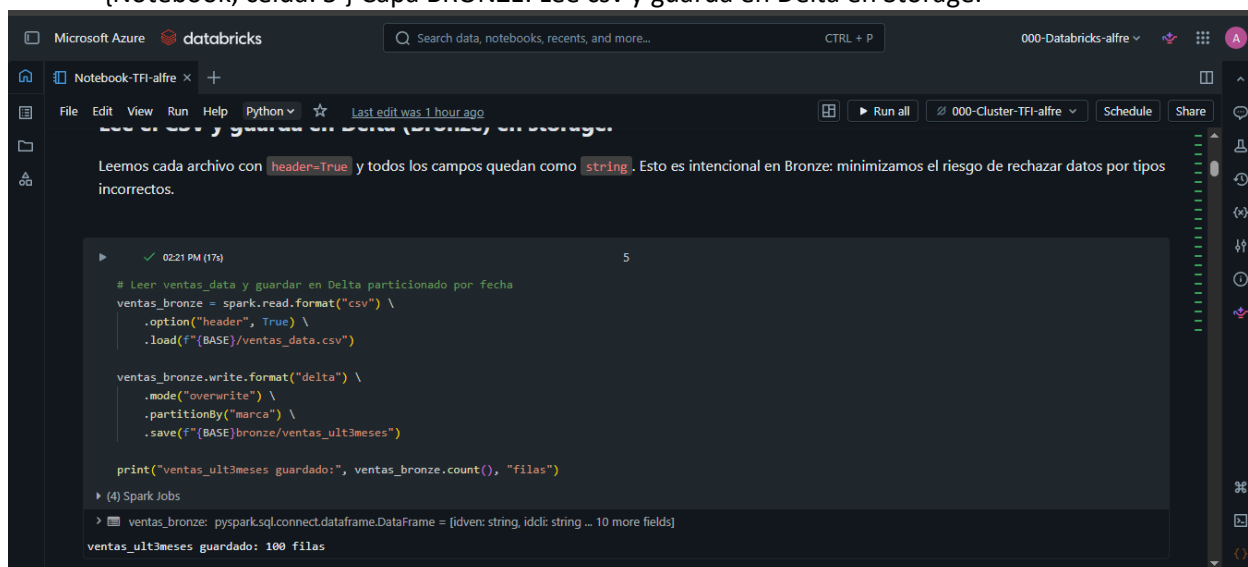
spark.conf.set(
    f"fs.azure.account.key.{storage_account}.blob.core.windows.net",
    key
)

BASE = f"wasbs://{container}@{storage_account}.blob.core.windows.net/consesionario1a/"

# Verificar que los archivos son accesibles
dbutils.fs.ls(BASE)

▶ (1) Spark Jobs
```

{Notebook, celda: 5 } Capa BRONZE. Lee csv y guarda en Delta en Storage.



The screenshot shows the Databricks interface for the same notebook. The code in cell 5 is as follows:

```
# Leer ventas_data y guardar en Delta particionado por fecha
ventas_bronze = spark.read.format("csv") \
    .option("header", True) \
    .load(f"{BASE}/ventas_data.csv")

ventas_bronze.write.format("delta") \
    .mode("overwrite") \
    .partitionBy("marca") \
    .save(f"{BASE}bronze/ventas_ult3meses")

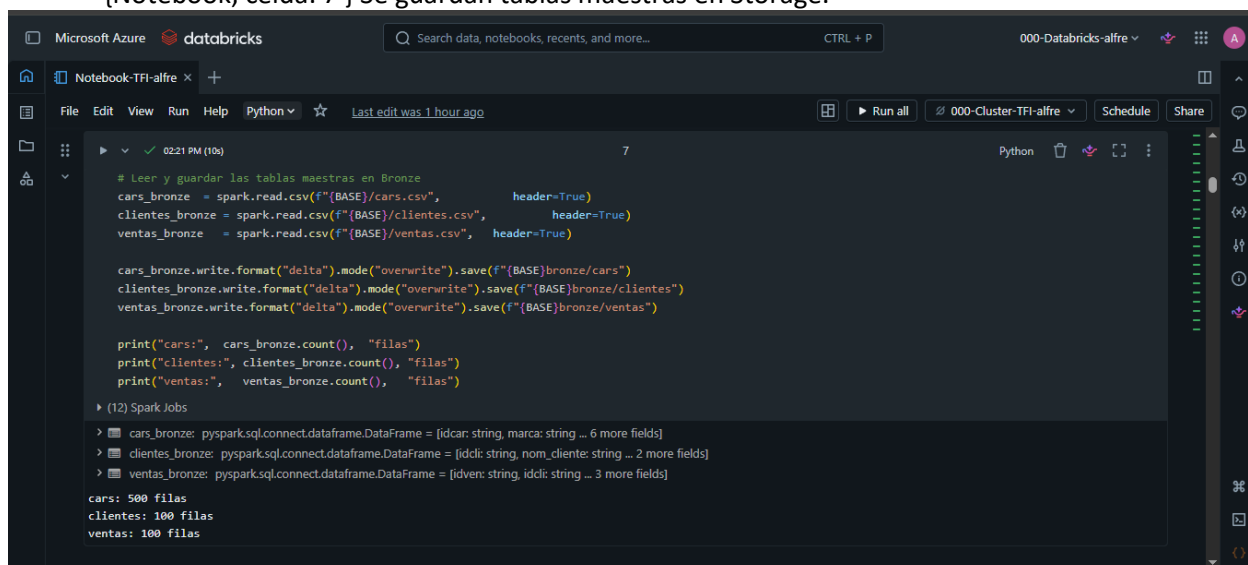
print("ventas_ult3meses guardado:", ventas_bronze.count(), "filas")

▶ (4) Spark Jobs
```

Below the code, the output shows:

```
ventas_bronze: pyspark.sql.connect.dataframe.DataFrame = [idven: string, idcli: string ... 10 more fields]
ventas_ult3meses guardado: 100 filas
```

{Notebook, celda: 7 } Se guardan tablas maestras en Storage.



The screenshot shows the Databricks interface for the same notebook. The code in cell 7 is as follows:

```
# Leer y guardar las tablas maestras en Bronze
cars_bronze = spark.read.csv(f"{BASE}/cars.csv", header=True)
clientes_bronze = spark.read.csv(f"{BASE}/clientes.csv", header=True)
ventas_bronze = spark.read.csv(f"{BASE}/ventas.csv", header=True)

cars_bronze.write.format("delta").mode("overwrite").save(f"{BASE}bronze/cars")
clientes_bronze.write.format("delta").mode("overwrite").save(f"{BASE}bronze/clientes")
ventas_bronze.write.format("delta").mode("overwrite").save(f"{BASE}bronze/ventas")

print("cars:", cars_bronze.count(), "filas")
print("clientes:", clientes_bronze.count(), "filas")
print("ventas:", ventas_bronze.count(), "filas")

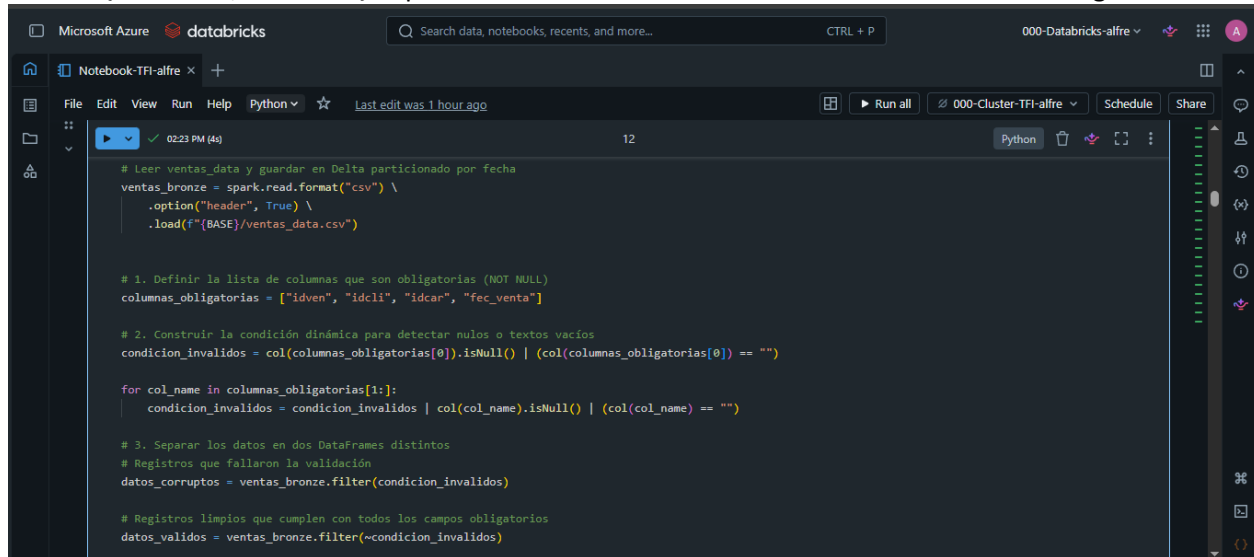
▶ (12) Spark Jobs
```

Below the code, the output shows:

```
cars_bronze: pyspark.sql.connect.dataframe.DataFrame = [idcar: string, marca: string ... 6 more fields]
clientes_bronze: pyspark.sql.connect.dataframe.DataFrame = [idcli: string, nom_cliente: string ... 2 more fields]
ventas_bronze: pyspark.sql.connect.dataframe.DataFrame = [idven: string, idcli: string ... 3 more fields]

cars: 500 filas
clientes: 100 filas
ventas: 100 filas
```


{Notebook, celda: 12 } Capa SILVER. Mecanismo de Calidad 1: Control de Datos obligatorios nonull



```
Microsoft Azure databricks
Search data, notebooks, recents, and more... CTRL + P
000-Databricks-alfre
Notebook-TFI-alfre
File Edit View Run Help Python Last edit was 1 hour ago
Run all 000-Cluster-TFI-alfre Schedule Share
02:23 PM (4s) 12 Python
# Leer ventas_data y guardar en Delta particionado por fecha
ventas_bronze = spark.read.format("csv") \
    .option("header", True) \
    .load(f"{BASE}/ventas_data.csv")

# 1. Definir la lista de columnas que son obligatorias (NOT NULL)
columnas_obligatorias = ["idven", "idcli", "idcar", "fec_venta"]

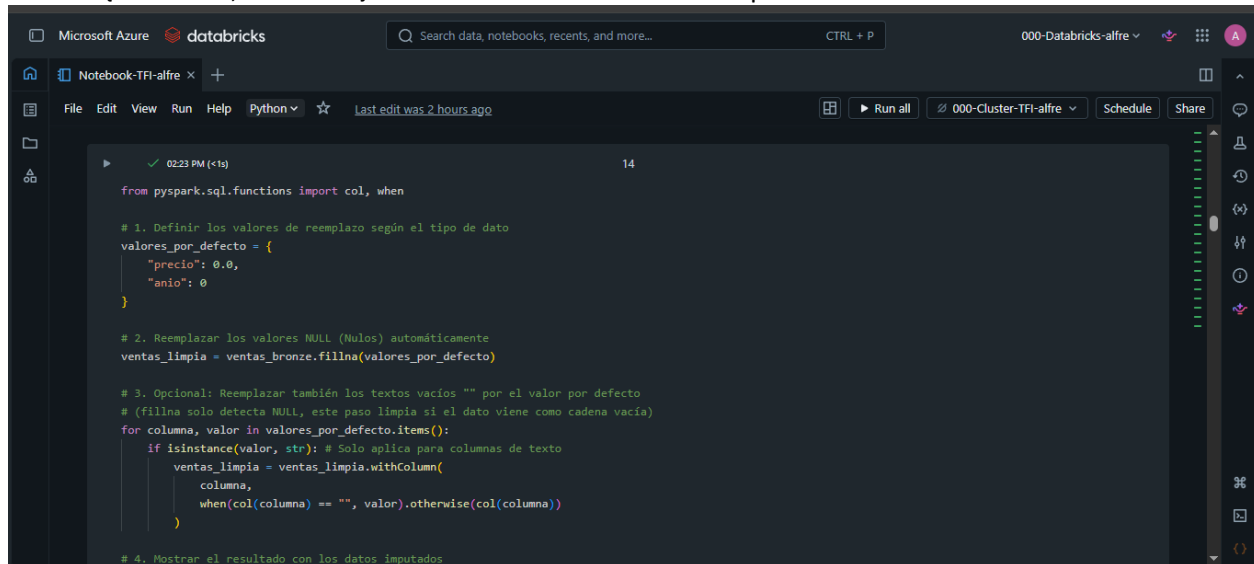
# 2. Construir la condición dinámica para detectar nulos o textos vacíos
condicion_invalidos = col(columnas_obligatorias[0]).isNull() | (col(columnas_obligatorias[0]) == "")

for col_name in columnas_obligatorias[1:]:
    condicion_invalidos = condicion_invalidos | col(col_name).isNull() | (col(col_name) == "")

# 3. Separar los datos en dos DataFrames distintos
# Registros que fallaron la validación
datos_corruptos = ventas_bronze.filter(condicion_invalidos)

# Registros limpios que cumplen con todos los campos obligatorios
datos_validos = ventas_bronze.filter(~condicion_invalidos)
```

{Notebook, celda: 14 } Mecanismo de Calidad 2: Reemplazo de valores nulos.



```
Microsoft Azure databricks
Search data, notebooks, recents, and more... CTRL + P
000-Databricks-alfre
Notebook-TFI-alfre
File Edit View Run Help Python Last edit was 2 hours ago
Run all 000-Cluster-TFI-alfre Schedule Share
02:23 PM (<1s) 14 Python
from pyspark.sql.functions import col, when

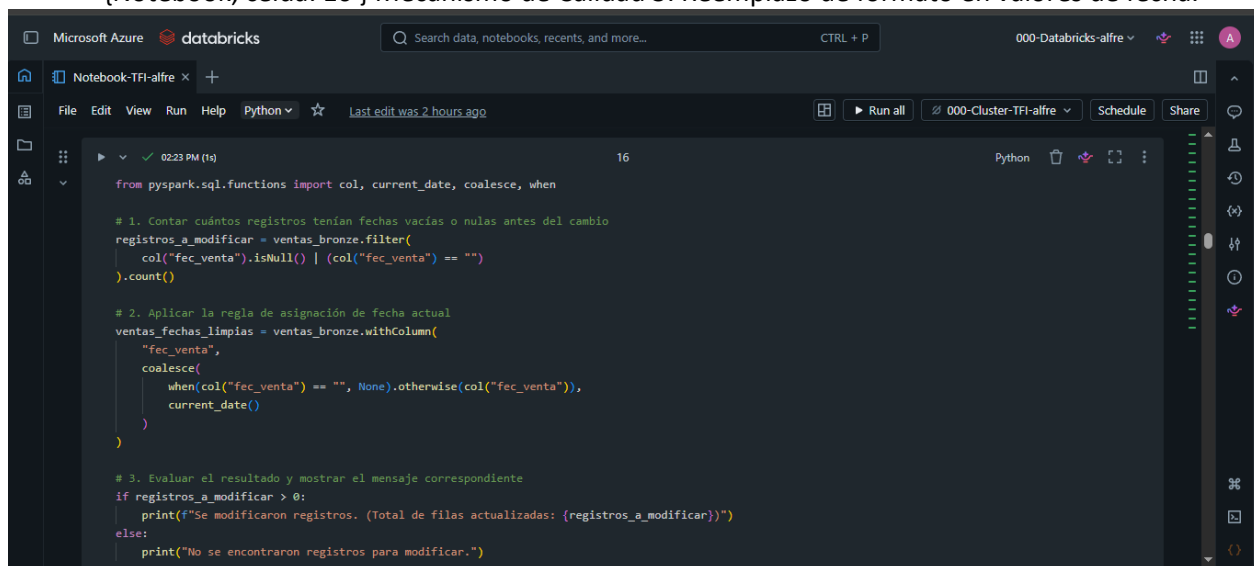
# 1. Definir los valores de reemplazo según el tipo de dato
valores_por_defecto = {
    "precio": 0.0,
    "anio": 0
}

# 2. Reemplazar los valores NULL (Nulos) automáticamente
ventas_limpia = ventas_bronze.fillna(valores_por_defecto)

# 3. Opcional: Reemplazar también los textos vacíos "" por el valor por defecto
# (fillna solo detecta NULL, este paso limpia si el dato viene como cadena vacía)
for columna, valor in valores_por_defecto.items():
    if isinstance(valor, str): # Solo aplica para columnas de texto
        ventas_limpia = ventas_limpia.withColumn(
            columna,
            when(col(columna) == "", valor).otherwise(col(columna))
        )

# 4. Mostrar el resultado con los datos imputados
```

{Notebook, celda: 16 } Mecanismo de Calidad 3: Reemplazo de formato en valores de fecha.



```
Microsoft Azure databricks
Search data, notebooks, recents, and more... CTRL + P
000-Databricks-alfre
Notebook-TFI-alfre
File Edit View Run Help Python Last edit was 2 hours ago
Run all 000-Cluster-TFI-alfre Schedule Share
02:23 PM (1s) 16 Python
from pyspark.sql.functions import col, current_date, coalesce, when

# 1. Contar cuántos registros tenían fechas vacías o nulas antes del cambio
registros_a_modificar = ventas_bronze.filter(
    col("fec_venta").isNull() | (col("fec_venta") == "")
).count()

# 2. Aplicar la regla de asignación de fecha actual
ventas_fechas_limpias = ventas_bronze.withColumn(
    "fec_venta",
    coalesce(
        when(col("fec_venta") == "", None).otherwise(col("fec_venta")),
        current_date()
    )
)

# 3. Evaluar el resultado y mostrar el mensaje correspondiente
if registros_a_modificar > 0:
    print(f"Se modificaron registros. (Total de filas actualizadas: {registros_a_modificar})")
else:
    print("No se encontraron registros para modificar.")
```

```

    ).count()

    # 2. Aplicar la regla de asignación de fecha actual
    ventas_fechas_limpias = ventas_bronze.withColumn(
        "fec_venta",
        coalesce(
            when(col("fec_venta") == "", None).otherwise(col("fec_venta")),
            current_date()
        )
    )

    # 3. Evaluar el resultado y mostrar el mensaje correspondiente
    if registros_a_modificar > 0:
        print(f"Se modificaron registros. (Total de filas actualizadas: {registros_a_modificar})")
    else:
        print("No se encontraron registros para modificar.")
    
```

(2) Spark Jobs

ventas_fechas_limpias: pyspark.sql.connect.dataframe.DataFrame = [idven: string, idcli: string ... 10 more fields]

No se encontraron registros para modificar.

{Notebook, celda: 18 } Limpieza: deduplicación y conversión de fecha.

```

    # Deduplicar por clave de ventas y tipar la fecha
    from pyspark.sql.functions import to_date

    ventas_clean = ventas_ult3meses \
        .dropDuplicates(["idven", "idcli", "idcar", "fec_venta"]) \
        .withColumn("fec_venta", to_date("fec_venta", "yyyy-MM-dd"))

    print("Filas después de limpieza:", ventas_clean.count())
    ventas_clean.printSchema()
    
```

(3) Spark Jobs

ventas_clean: pyspark.sql.connect.dataframe.DataFrame = [idven: string, idcli: string ... 10 more fields]

Filas después de limpieza: 100

```

root
 |-- idven: string (nullable = true)
 |-- idcli: string (nullable = true)
 |-- idcar: string (nullable = true)
 |-- fec_venta: date (nullable = true)
 |-- nom_vendedor: string (nullable = true)
 |-- nom_cliente: string (nullable = true)
 |-- marca: string (nullable = true)
    
```

{Notebook, celda: 20 } Hacemos inner join contra ventas, clientes y cars. El resultado es una sola tabla con toda la información necesaria para análisis.

```

    # Join de ventas con las tres tablas maestras
    ventas_silver = ventas_clean \
        .join(clientes, on="idcli", how="inner") \
        .join(cars, on="idcar", how="inner")

    print("Filas en tabla Silver:", ventas_silver.count())
    ventas_silver.printSchema()
    ventas_silver.display()
    
```

(9) Spark Jobs

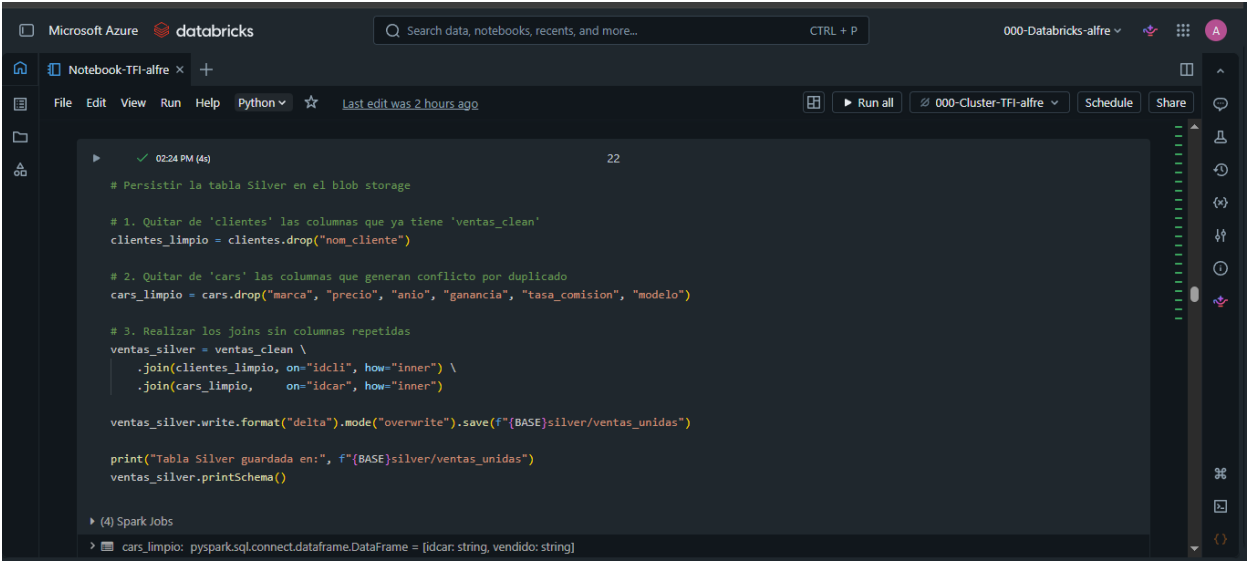
ventas_silver: pyspark.sql.connect.dataframe.DataFrame = [idcar: string, idcli: string ... 20 more fields]

Filas en tabla Silver: 100

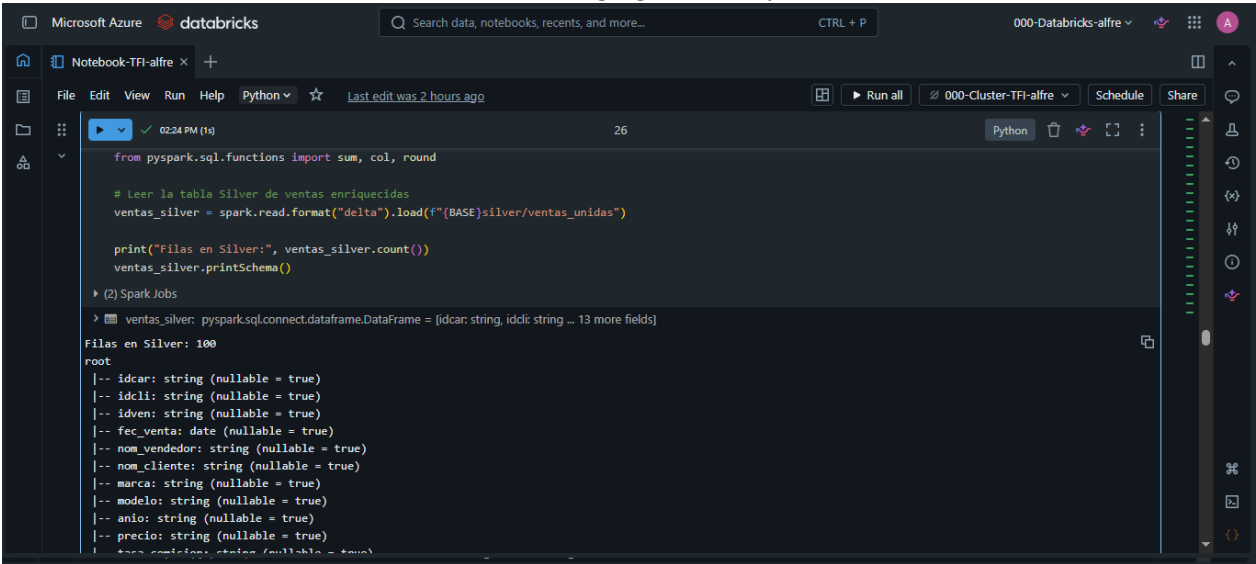
```

root
 |-- idcar: string (nullable = true)
 |-- idcli: string (nullable = true)
 |-- idven: string (nullable = true)
 |-- fec_venta: date (nullable = true)
 |-- nom_vendedor: string (nullable = true)
 |-- nom_cliente: string (nullable = true)
 |-- marca: string (nullable = true)
 |-- modelo: string (nullable = true)
    
```

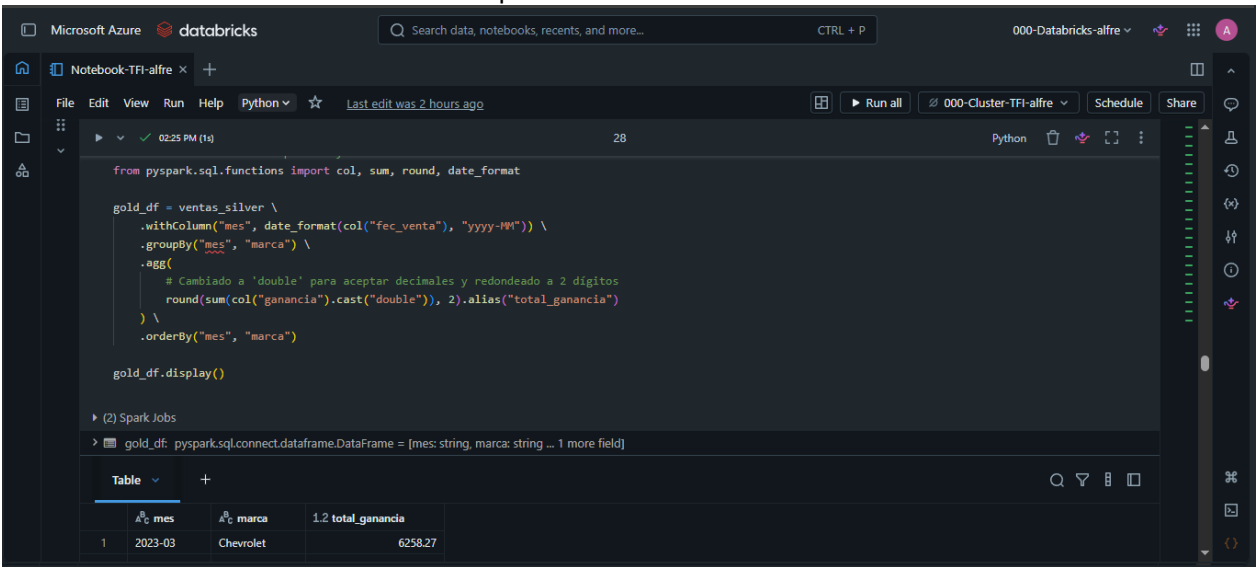
{Notebook, celda: 22 } Se Guarda la tabla Silver en Delta.



{Notebook, celda: 26 } Capa GOLD. Agregaciones para análisis Genera tablas analíticas optimizadas a partir de los datos Silver. El resultado es una tabla de hechos agregados lista para BI.



{Notebook, celda: 28 } Agrupamos por MES y MARCA (no por ID, para que la tabla Gold sea autoexplicativa) y calculamos el monto total de cada marca por mes.



{Notebook, celda: 30 } Se Guarda la tabla Gold en Delta.

Notebook-TFI-alfre

File Edit View Run Help Python Last edit was 2 hours ago

12 rows | 1.15s runtime Refreshed 2 hours ago

Guardar la tabla Gold en Delta

```
# Persistir la tabla Gold en el blob storage
gold_df.write.format("delta").mode("overwrite").save(f"{BASE}gold/ventas_x_mes_marca")

print("Tabla Gold guardada en:", f"{BASE}gold/ventas_x_mes_marca")
```

(4) Spark Jobs

Tabla Gold guardada en: wasbs://000container@000storage.blob.core.windows.net/consesionaria/gold/ventas_x_mes_marca

Consulta SQL de validación

1	2023-03	Chevrolet	6258.27
---	---------	-----------	---------

{Notebook, celda: 35 } Pedido de Gerencia: Mostrar total de ganancia por mes. Mostrar gráfico representativo.

Microsoft Azure databricks

Search data, notebooks, recents, and more... CTRL + P

000-Databricks-alfre

Notebook-TFI-alfre

File Edit View Run Help Python Last edit was 2 hours ago

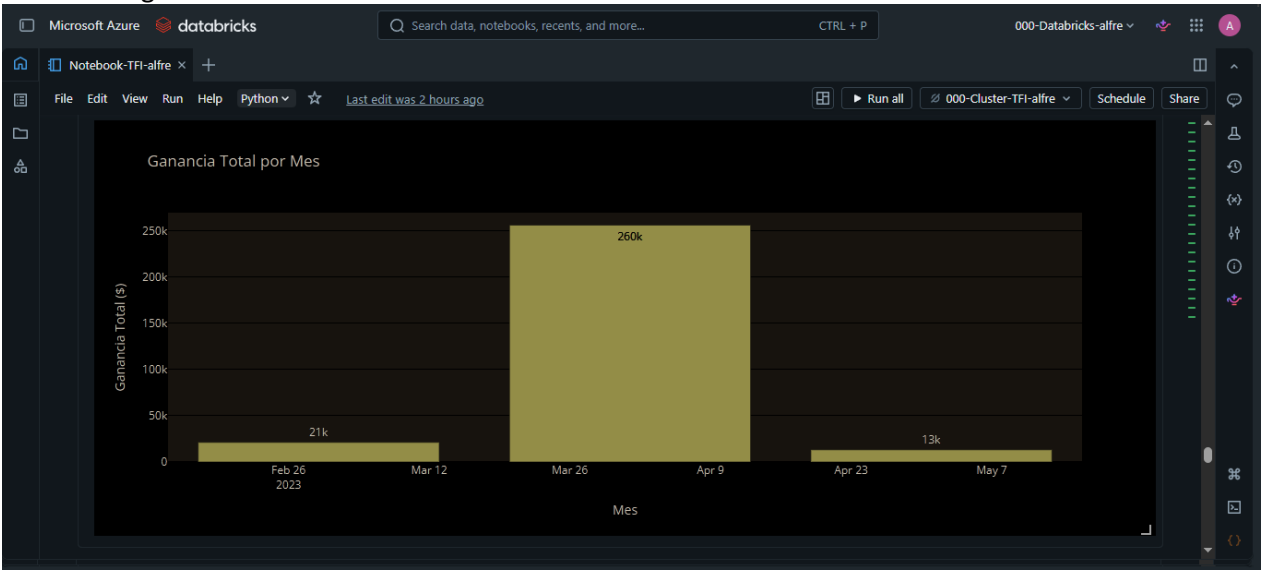
```
SELECT
    mes,
    ROUND(SUM(total_ganancia), 2) AS ganancia_total_mes,
    ROUND(AVG(total_ganancia), 2) AS promedio_ganancia_por_mes
FROM gold_sql
GROUP BY mes
ORDER BY mes;
```

(5) Spark Jobs

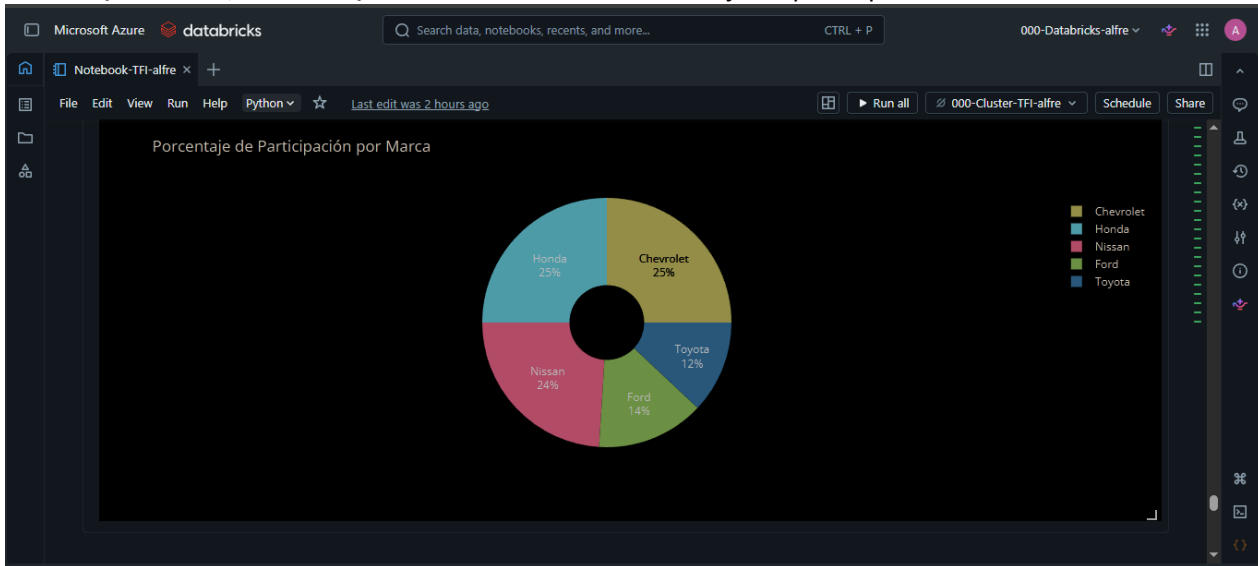
_sqlid: pyspark.sql.connect.dataframe.DataFrame = (mes: string, ganancia_total_mes: double ... 1 more field)

Table	mes	1.2 ganancia_total_mes	1.2 promedio_ganancia_por_mes
1	2023-03	21151.26	5287.82
2	2023-04	255847.61	51169.52
3	2023-05	13159.6	4386.53

{Notebook, celda: 38 } GRAFICO DE BARRAS: interactivo donde cada barra representa un mes y su altura muestra el total de las ganancias acumuladas.



{Notebook, celda: 42 } GRAFICO DE TORTA: Porcentaje de participación de cada marca en los ultimos 3 meses.



{Notebook, celda: 44 } GRAFICO DE LINEAS: Tendencia, evolución de las ganancias mes a mes por marca.



REQUERIMIENTOS PUNTUALES DE CONSIGNA DE TRABAJO FINAL:

1. ORIGENES DE DATOS:

Los archivos que se van a utilizar son:

ventas_data_suc.csv

ventas_data.csv

cliente.csv

ventas.csv

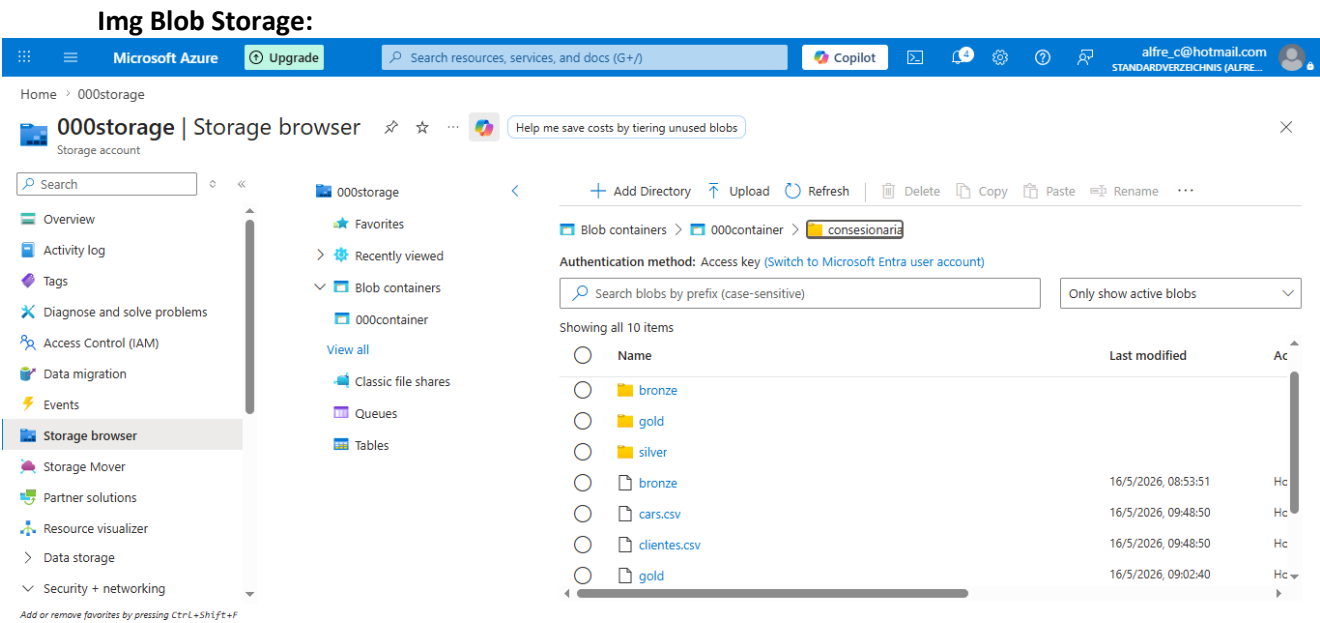
cars.csv

Todos almacenados en Blob Storage 000storage

El archivo ventas_data_suc.csv es extraído mediante un pipeline (000_p_url_storage) de un repositorio Github (alfre309/tfi-alfre) que simula una API de la sucursal de la empresa.

[url: https://raw.githubusercontent.com/alfre309/tfi-alfre/refs/heads/main/ventas_data.csv]

El archivo ventas_data.csv (suponemos) es generado por el sistema de la empresa y depositado en el mismo Blob Storage.



2. TRANSFORMACIONES QUE SE VAN A HACER:

- Joins entre archivos fuentes: clientes.csv, ventas.csv y cars.csv - {Notebook, celda 19 y 20}
- Eliminación de duplicados en capa Silver. - {Notebook, celda 17 y 18}
- Reconfiguración del campo fecha. - {Notebook, celda 17 y 18}
- Conversiones de CSV a Parquet/Delta. - {Notebook, celda 23 y 24}
- Almacenado de tablas en capas Bronze, Silver, Gold en Storage. - {Notebook, a partir de celda 4}

3. MECANISMOS DE CALIDAD DE DATOS:

- MECANISMO DE CALIDAD 1: Control de Datos obligatorios nonull - {Notebook, celdas 11 y 12}
- MECANISMO DE CALIDAD 2: Reemplazo de valores nulos - {Notebook, celdas 13 y 14}
- MECANISMO DE CALIDAD 3: Reemplazo de formato en valores de fecha - {Notebook, celdas 15 y 16}

4. PRODUCCIÓN DE TABLAS GOLD

- Se Genera tablas analíticas optimizadas a partir de los datos Silver para cualquier tipo de análisis. {Notebook, a partir de celda 25}

5. EL INFORME TÉCNICO DEL PROCESO EMPIEZA EN PÁGINA 1.